



UNIVERSIDAD
DE GRANADA



Técnicas de los Sistemas Inteligentes

Grado en Informática

Curso 2025-26. Práctica 1

Técnicas de Búsqueda Heurística

Pablo Mesejo Santiago y David Criado Ramón

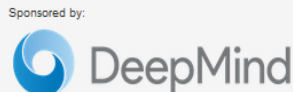
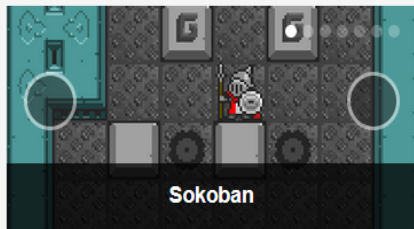
**Departamento de Ciencias de la
Computación e Inteligencia Artificial**

<http://decsai.ugr.es>

General Video Game AI (GVGAI)

- Website: <https://github.com/GAIGResearch/GVGAI>
- GVGAI: una competición para agentes de IA para
 - Jugar a videojuegos
 - Generación de contenido (mapas de niveles y reglas de juego)
- Los agentes desarrollados se evalúan sobre videojuegos no vistos previamente
- +160 Videojuegos
- Los juegos se describen en el lenguaje VGDL (Video Game Description Language)

Schaul, T., 2013, August. A video game description language for model-based or interactive learning. In 2013 IEEE Conference on Computational Intelligence in Games (CIG) (pp. 1-8). IEEE.



Welcome to the General Video Game AI Competition webpage. The GVG-AI Competition explores the problem of creating controllers for general video game playing. How would you create a single agent that is able to play any game it is given? Could you program an agent that is able to play a wide variety of games, without knowing which games are to be played? Can you create an automatic level generation that designs levels for any game is given?

In this website, you will be able to participate in the General Video Game AI Competition. You can now download the starter kit for the competition and submit your controller to be included in the rankings. For any question contact us.

Tiempo Real

- Tiempo límite reacción: 40 ms
- Si devuelves acción en [40,50] ms, se aplica NIL
- Si devuelves acción en > 50 ms, pierdes juego. Descalificado

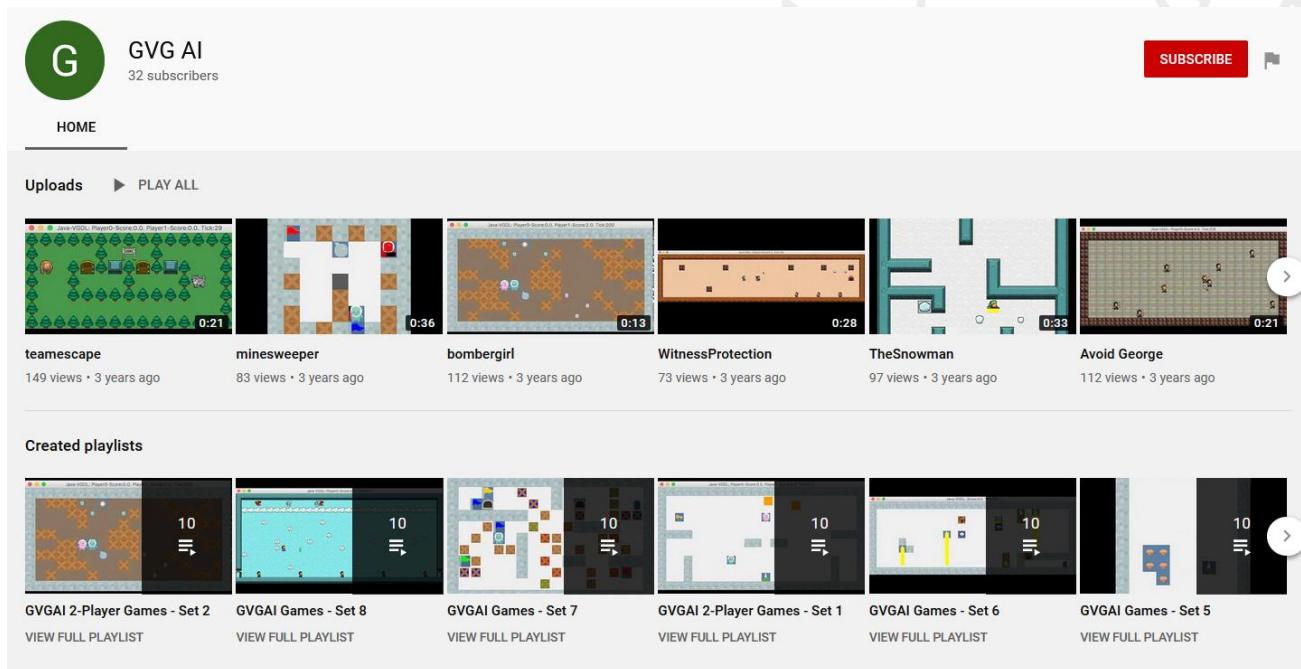
Descripciones de los videojuegos y vídeos de ejemplo

GVGAI CHANNEL

<https://www.youtube.com/channel/UCMFCfXipQT55IK6R504naUQ>

Tipos de juegos

- Un jugador / varios
- Total / parcialmente observables
- Puzzles vs Carreras
- Acción / Aventuras
- Recolectar, disparar, navegar
- Cooperativos / Competitivos



The screenshot shows the GVGAI YouTube channel page. At the top, there is a green circular profile picture with a white 'G' and the text 'GVG AI' and '32 subscribers'. Below this is a 'HOME' tab. The main content area is titled 'Uploads' and 'PLAY ALL'. It displays a grid of video thumbnails for various games: 'teamscape' (149 views, 3 years ago), 'minesweeper' (83 views, 3 years ago), 'bombergirl' (112 views, 3 years ago), 'WitnessProtection' (73 views, 3 years ago), 'TheSnowman' (97 views, 3 years ago), and 'Avoid George' (112 views, 3 years ago). Below the grid, there is a section titled 'Created playlists' showing several playlists with thumbnails and titles like 'GVGAI 2-Player Games - Set 2', 'GVGAI Games - Set 8', 'GVGAI Games - Set 7', 'GVGAI 2-Player Games - Set 1', 'GVGAI Games - Set 6', and 'GVGAI Games - Set 5'. Each playlist has a 'VIEW FULL PLAYLIST' link.

Varios Tracks: está concebido para probar distintas técnicas de IA en videojuegos tanto para jugar a videojuegos como para generar contenido.

Jugar

- **Single Player Track**
- Multiple Player Track
- Single Player Learning Track

Generar contenido

- Level Generation Track
- Rule Generation Track
- Game Design Track

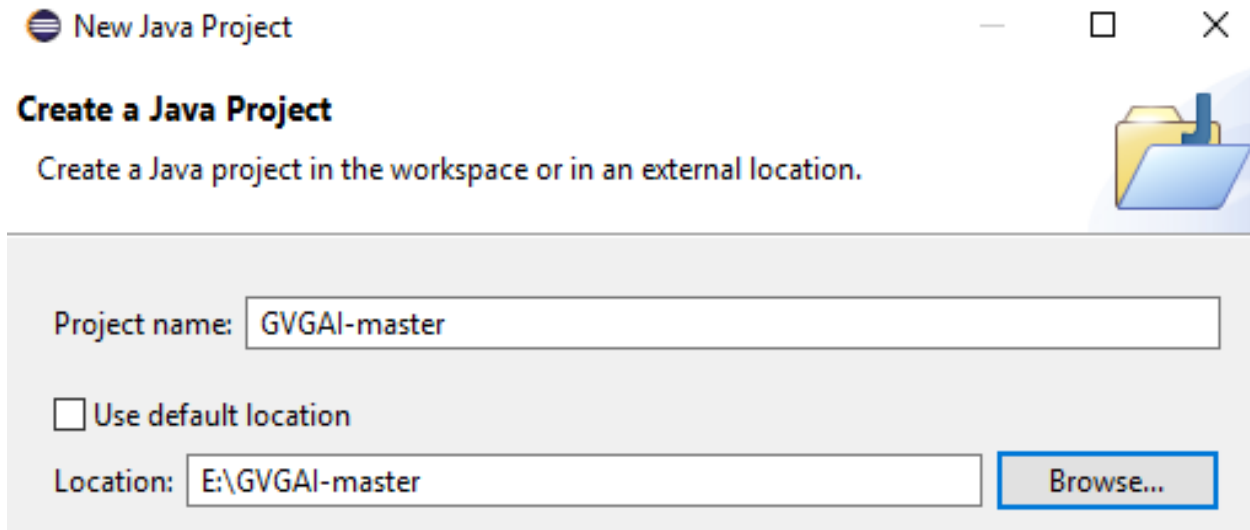
Instalación del GVG Framework

- Descargad el zip directamente desde
<https://github.com/GAIGResearch/GVGAI/archive/master.zip>
<https://pradogrado2526.ugr.es/mod/resource/view.php?id=293431>
- Enlace de posible utilidad o interés:
<https://gaigresearch.github.io/gvgaibook/>



Instalación del GVG Framework con ECLIPSE (hay que tener instalado Java y Eclipse)

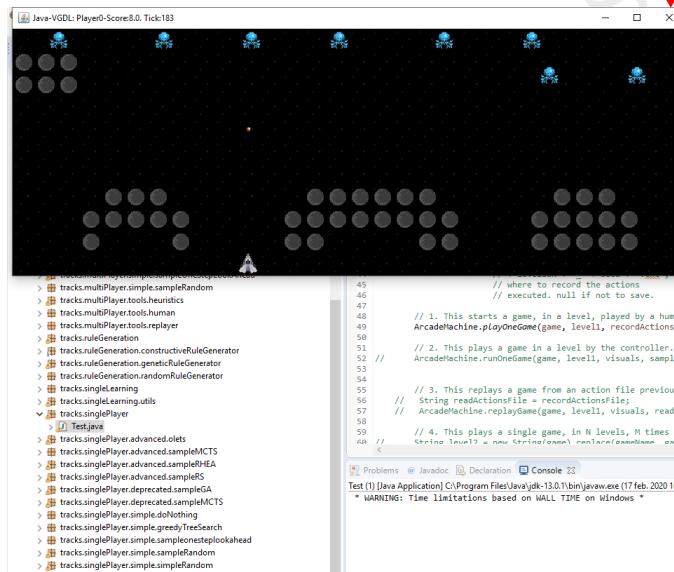
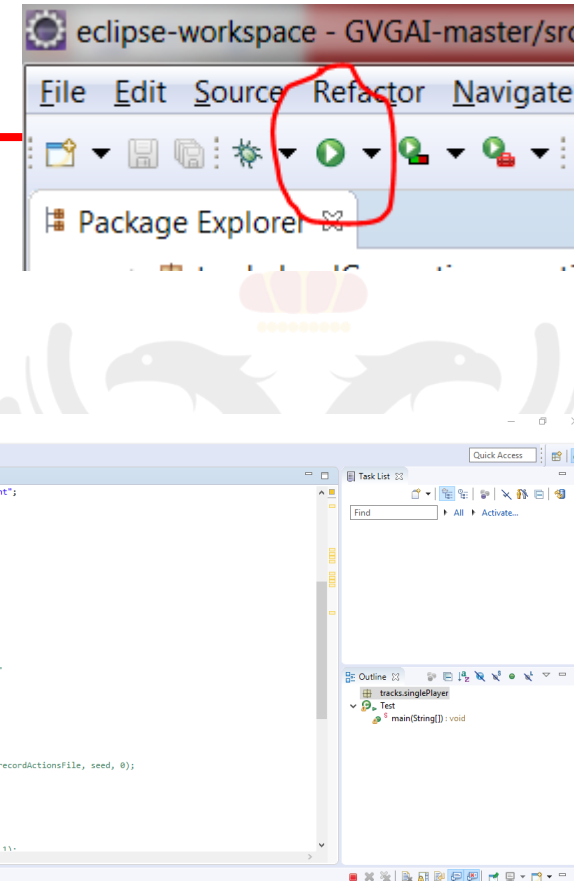
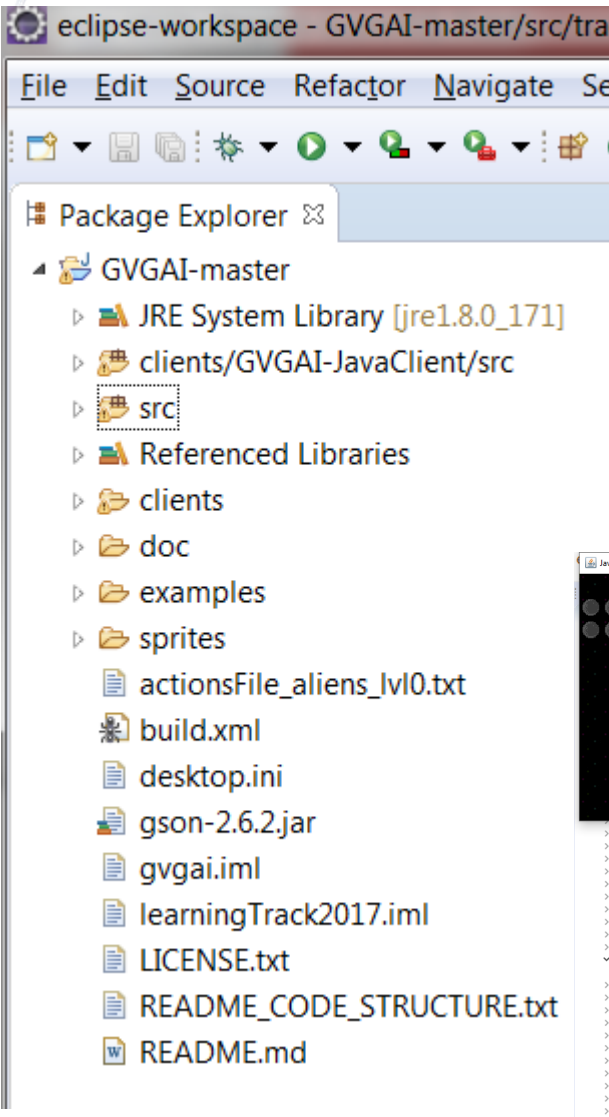
- Descomprimir en un <directorio>
- Abrir Eclipse.
- File → New → Java Project
- Untick 'default location' y seleccionar <directorio> en Location → Finish



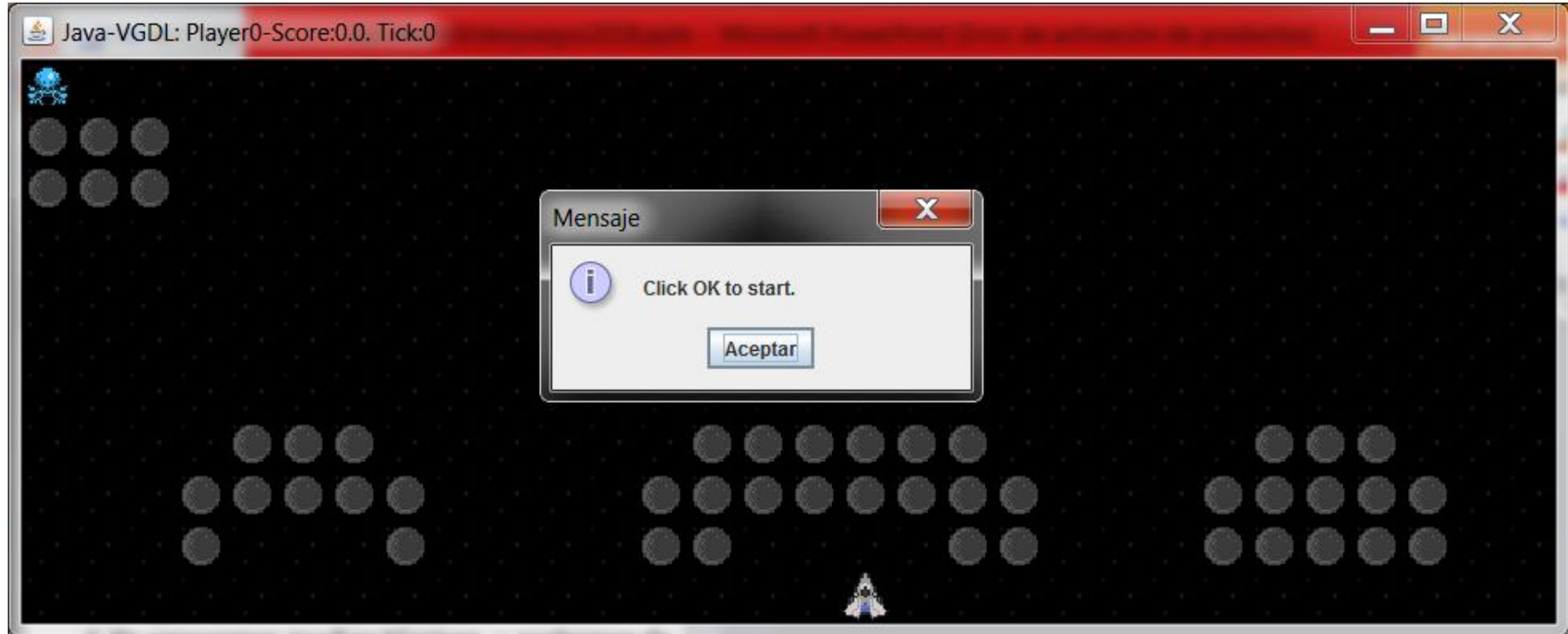
- Desmarcad la casilla 'Create module-info.java file' (en caso de que esté marcada).

Ejecución del GVG Framework

- En Package Explorer, navegar hasta
 - Src.tracks.singlePlayer → Test.java
 - Ejecutar Test.java



Ejecución del GVG Framework



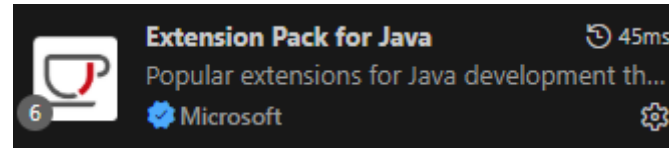
- Por defecto el entorno de juego carga el juego "0" (gameIdx) y en modo jugador humano (*ArcadeMachine.playOneGame*).

La estructura del código y documentación básica está en

<https://github.com/EssexUniversityMCTS/gvgai/wiki/Code-Structure>

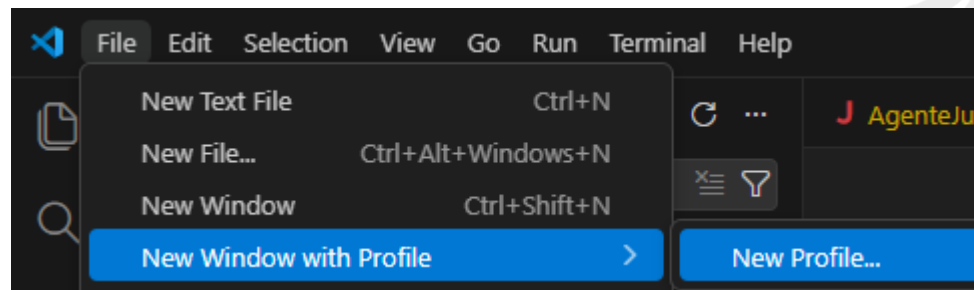
Instalación del GVG Framework con VSCODE (hay que tener instalado Java y VSCODE)

- Descomprimir en un <directorio> y ABRIR VSCODE
- Es necesario tan solo utilizar la extensión "**Extension Pack for Java**".



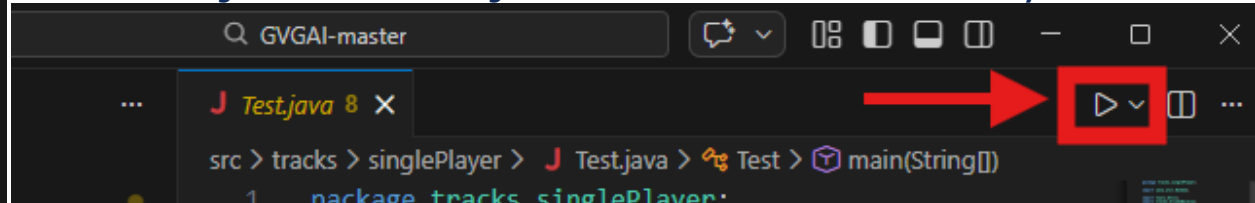
- Tras ello debe abrir la carpeta <directorio> donde ha descomprimido GVGAI-MASTER en "**File > Open Folder...**" y el entorno estará listo.

NOTA: Combinar múltiples extensiones de Java diferentes suele dar problemas. Si ya tiene otra instalación distinta, y no quiere modificar su entorno, considere crear un perfil nuevo para esta asignatura en "File > New Window with Profile > New Profile..."

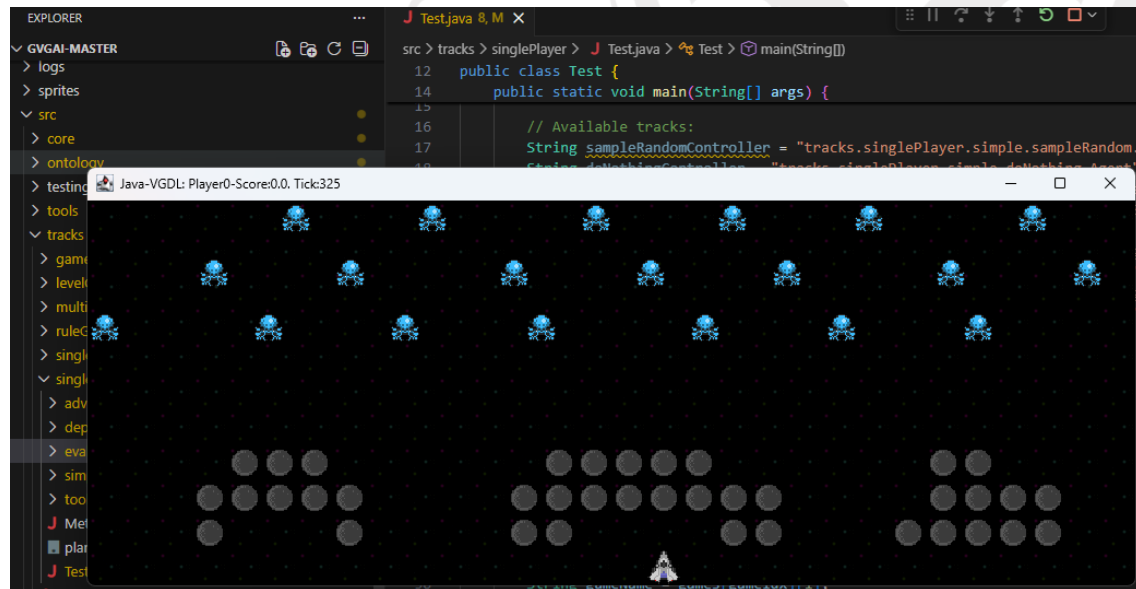


Ejecución del GVG Framework

- En explorer (lateral izquierdo) navegar hasta
 - Src.tracks.singlePlayer → Test.java
 - Ejecutar Test.java usando el botón Play



- Una vez ejecutado por primera vez, basta con pulsar el botón desde cualquier otro archivo .java que no contenga un main para volver a ejecutar.



Modificación del GVG Framework

src.tracks.singlePlayer.Test.java

```
//Load available games
String spGamesCollection = "examples/all_games_sp.csv";
String[][] games = Utils.readGames(spGamesCollection);

//Game settings
boolean visuals = true;
int seed = new Random().nextInt();

// Game and level to play
int gameIdx = 0;
int levelIdx = 0; // level names from 0 to 4 (game_lvlN.txt).
String gameName = games[gameIdx][1];
String game = games[gameIdx][0];
String level1 = game.replace(gameName, gameName + "_lvl" + levelIdx);

String recordActionsFile = null; // "actions_" + games[gameIdx] + "_lvl"
    // + levelIdx + "_" + seed + ".txt";
    // where to record the actions
    // executed. null if not to save.
```

Consultad este fichero.
Hay numerosos juegos
disponibles para
SinglePlayer.

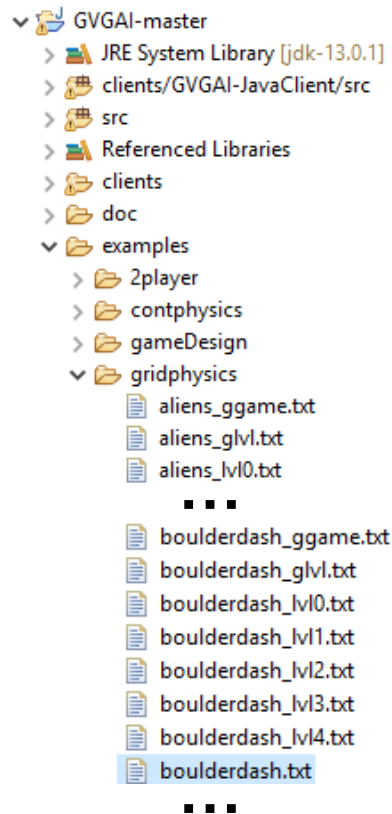
int gameIdx = 0; Cambiad este valor y ejecutad para ver otros juegos.



¿Cómo podemos saber cómo se definen, por ejemplo, los mapas del juego BoulderDash (id 11)?

(1) Abrir all_games_sp.csv → (2) Abrir la ruta relativa a boulderdash.txt

	A	B	C	D
1	0,examples/gridphysics/aliens.txt			
2	1,examples/gridphysics/angelsdemons.txt			
3	2,examples/gridphysics/assemblyline.txt			
4	3,examples/gridphysics/avoidgeorge.txt			
5	4,examples/gridphysics/bait.txt			
6	5,examples/gridphysics/beltmanager.txt			
7	6,examples/gridphysics/blacksmoke.txt			
8	7,examples/gridphysics/boloadventures.txt			
9	8,examples/gridphysics/bomber.txt			
10	9,examples/gridphysics/bomberman.txt			
11	10,examples/gridphysics/boulcherdash.txt			
12	11,examples/gridphysics/boulderdash.txt			
13	12,examples/gridphysics/brainman.txt			
14	13,examples/gridphysics/butterflies.txt			
15	14,examples/gridphysics/cakybaky.txt			
16	15,examples/gridphysics/camelRace.txt			
17	16,examples/gridphysics/catapults.txt			
18	17,examples/gridphysics/chainreaction.txt			



(3) Ver en boulderdash.txt cómo se definen los objetos del mapa

```

1 BasicGame
2   SpriteSet
3     background > Immovable img=oryx/backBlack hidden=True
4     wall > Immovable autotiling=true img=oryx/dirtWall
5     sword > Flicker color=LIGHTGRAY limit=1 singleton=True img=oryx/pickaxe
6     dirt > Immovable color=BROWN img=oryx/backLBrown
7     exitdoor > Door color=GREEN img=oryx/door2
8     diamond > Resource color=YELLOW limit=10 shrinkfactor=0.75 img=oryx/diamond3
9     boulder > Missile orientation=DOWN color=GRAY speed=0.2 img=oryx/mineral1
10    moving >
11      avatar > ShootAvatar stype=sword frameRate=8 img=oryx/spelunky
12      enemy > RandomNPC cons=1
13        crab > color=RED img=oryx/scorpion2
14        butterfly > color=PINK img=oryx/bat2
15
16  LevelMapping
17    . > background dirt
18    - > background
19    e > background exitdoor
20    o > background boulder
21    x > background diamond
22    c > background crab
23    b > background butterfly
24    A > background avatar
25
26  InteractionSet
27    dirt avatar sword > killSprite
28    diamond avatar > collectResource scoreChange=2
29    moving wall boulder > stepBack
30
31    avatar boulder > killIfFromAbove scoreChange=-1
32    avatar butterfly crab > killSprite scoreChange=-1
33
34    boulder dirt wall diamond boulder > stepBack
35
36    enemy dirt diamond > stepBack
37
38    crab butterfly > killSprite
39    butterfly crab > transformTo stype=diamond scoreChange=1
40    exitdoor avatar > killIfOtherHasMore resource=diamond limit=9
41
42  TerminationSet
43    SpriteCounter stype=avatar limit=0 win=False
44    SpriteCounter stype=exitdoor limit=0 win=True
45

```

¿Cómo podemos saber cómo se definen, por ejemplo, los mapas del juego BoulderDash (id 11)?

(3) Ver en boulderdash.txt cómo se definen los objetos del mapa



(4) Ver el mapa del nivel 0 de BoulderDash y cómo se visualiza en el juego en la práctica

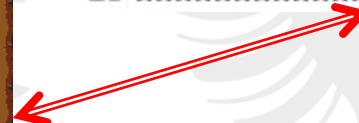
LevelMapping

```
. > background dirt
- > background
e > background exitdoor
o > background boulder
x > background diamond
c > background crab
b > background butterfly
A > background avatar
```



boulderdash_lvl0.txt

```
1 | ~~~~~
2 | w...o.xx.o.....o..xox..w
3 | w...ooooo.....o..o...w
4 | w...xxx.....o.oxoo.ow
5 | wx.....oxo...oow
6 | ~~~~~
7 | wb-...co.....wxxw
8 | w--.....Ao...o...wxxw
9 | wooo.....-...w..w
10 | w.....x...wwwx-x.oow..w
11 | wc--.....x..ooxxo-...w..w
12 | w---...e.....b-----w
13 | ~~~~~
```



Single Player Planning: GVGA proporciona agentes básicos para jugar (llamados "controllers") que pueden intercambiarse fácilmente.

```
// Available tracks:
String sampleRandomController = "tracks.singlePlayer.simple.sampleRandom.Agent";
String doNothingController = "tracks.singlePlayer.simple.doNothing.Agent";
String sampleOneStepController = "tracks.singlePlayer.simple.sampleonesteplookahead.Agent";
String sampleFlatMCTSController = "tracks.singlePlayer.simple.greedyTreeSearch.Agent";

String sampleMCTSController = "tracks.singlePlayer.advanced.sampleMCTS.Agent";
String sampleRSController = "tracks.singlePlayer.advanced.sampleRS.Agent";
String sampleRHEAController = "tracks.singlePlayer.advanced.sampleRHEA.Agent";
String sampleOLETSController = "tracks.singlePlayer.advanced.olets.Agent";
```

- Técnicas:
 - Aleatorio
 - No hacer nada
 - Avanzar una etapa (greedy)
 - Algoritmo genético
 - MCTS (Monte Carlo Tree Search)
 - ---



Experimentar con un agente deliberativo (single player planning agent)

1. Encontrar la clase `src.tracks.singlePlayer.Test.java`
2. Asignar `gameIdx` y `levelIdx` con distintos valores

3. Seleccionar modo de ejecución:

1. Jugar como humano:

```
// 1. This starts a game, in a level, played by a human.
```

```
ArcadeMachine.playOneGame(game, level1, recordActionsFile, seed);
```

2. Jugar como IA (con un "controller"): Usar (o crear) una variable `String` conteniendo el path de la clase de un agente (pre)diseñado.

```
String sampleMCTSController =  
"tracks.singlePlayer.advanced.sampleMCTS.Agent";
```

```
...
```

```
// 2. This plays a game in a level by the controller.
```

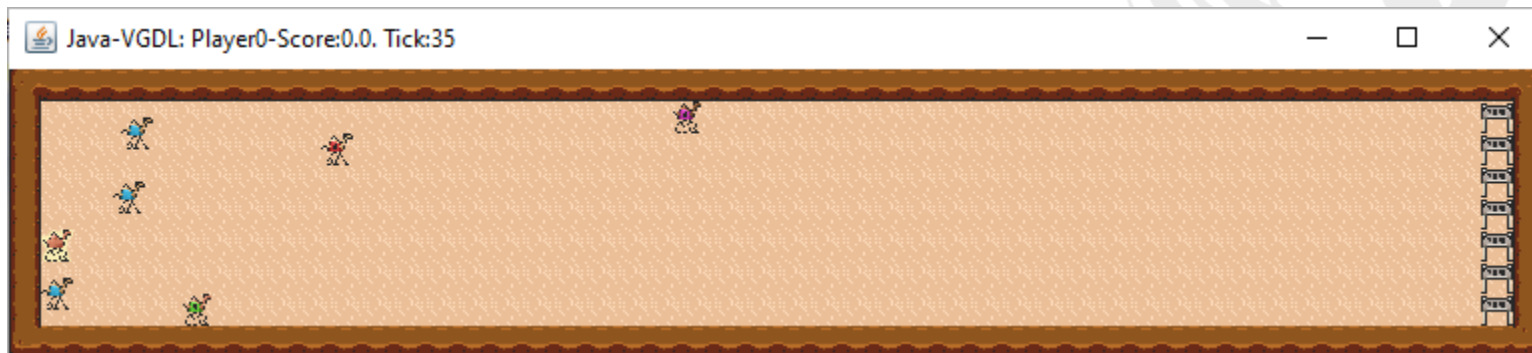
```
ArcadeMachine.runOneGame(game, level1, visuals,  
sampleMCTSController, recordActionsFile, seed, 0);
```

4. Run Test.java

Ejemplo de agente deliberativo para Camel Race

Escogemos el juego de **Carrera de Camellos** (int gameldx = 15) y el **primer nivel de juego** (int levelldx = 0).

Queremos que el agente escoja el portal de salida más cercano, y luego, **de modo voraz, escoja la acción que le acerque más al portal**. Es decir, queremos escoger en todo momento, y de modo inmediato, la acción más conveniente para alcanzar el objetivo de llegar al destino y ganar la carrera.



Código de ejemplo:

`src/tracks/singlePlayer/evaluacion/src_APELLIDO1_APELLIDO2_NOMBRE/myAgent_Camel.java`

```
package tracks.singlePlayer.evaluacion.src_APELLIDO1_APELLIDO2_NOMBRE;
```

```
public class myAgent_Camel extends AbstractPlayer{
```

```
// Aquí van las variables de estado del agente
```

```
// Constructor del agente
```

```
public myAgent_Camel(StateObservation stateObs, ElapsedCpuTimer elapsedTimer) {  
    // ...  
}
```

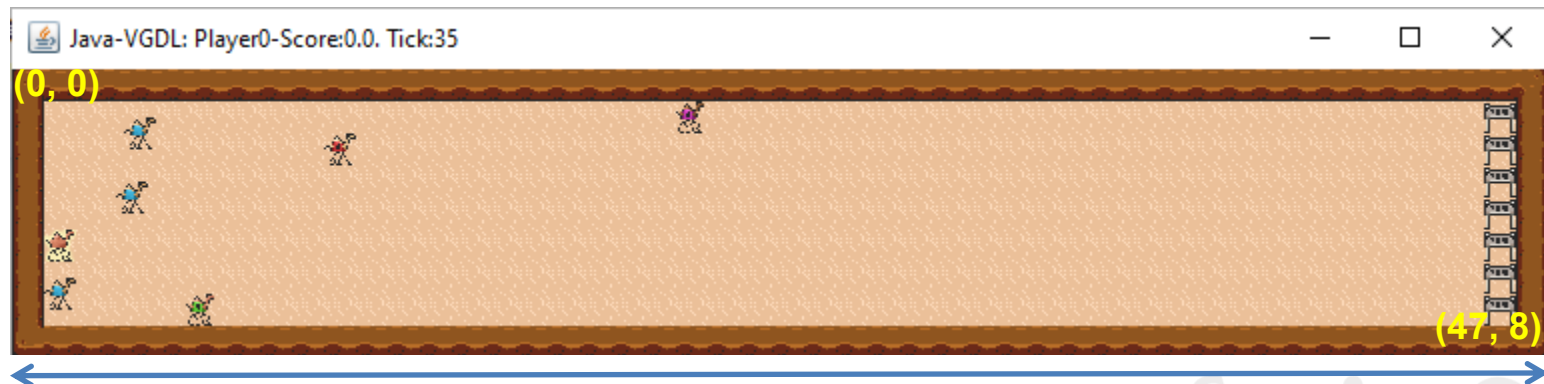
```
// Método act
```

```
@Override
```

```
public ACTIONS act(StateObservation stateObs, ElapsedCpuTimer elapsedTimer) {  
    // Se llama en cada tick y es el encargado de utilizar el estado interno  
    // del agente y la información de los sensores (stateObs) para decidir la  
    // siguiente acción (Types.ACTIONS)  
    // ...  
}
```

Código de ejemplo: myAgent_Camel.java

getWorldDimension().height : 144
getObservationGrid()[0].length : 9



stateObs.getWorldDimension().width : 768 || stateObs.getObservationGrid().length : 48

```
Vector2d fescala;  
Vector2d portal;
```

```
/**  
 * initialize all variables for the agent  
 * @param stateObs Observation of the current state.  
 * @param elapsedTimer Timer when the action returned is due.  
 */  
public myAgent_Camel(StateObservation stateObs, ElapsedCpuTimer elapsedTimer){  
    //Calculamos el factor de escala entre mundos (píxeles -> grid)  
    fescala = new Vector2d(stateObs.getWorldDimension().width / stateObs.getObservationGrid().length ,  
        stateObs.getWorldDimension().height / stateObs.getObservationGrid()[0].length);  
    //Se crea una lista de observaciones de portales, ordenada por cercanía al avatar  
    ArrayList<Observation>[] posiciones = stateObs.getPortalsPositions(stateObs.getAvatarPosition());  
    //Seleccionamos el portal mas proximo  
    portal = posiciones[0].get(0).position;  
    portal.x = Math.floor(portal.x / fescala.x);  
    portal.y = Math.floor(portal.y / fescala.y);  
}
```

Píxeles del tablero/mundo

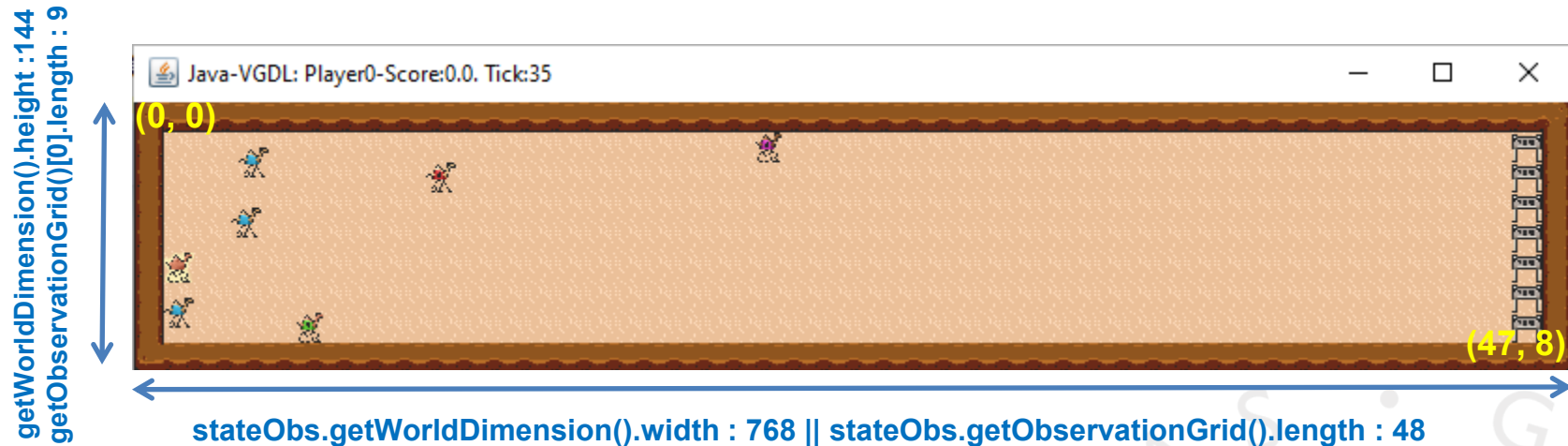
Posiciones del grid

Para una alternativa de cara a obtener este factor de escala, revísese `stateObs.getBlockSize();`

Número de píxeles de cada celda (posición del grid)

Posición de los portales en coordenadas píxel

Del primer tipo de portal (podría haber varios) cogemos el primer elemento (el más próximo)



Del mismo modo que podemos acceder a los portales, podemos acceder a todo tipo de recursos/objetos:

Para obtener la posición de los enemigos, si estos existiesen

Para obtener la posición de los muros y otros obstáculos

Para obtener la posición de las gemas en Boulderdash o las capas en LabyrinthDual

Función	Descripción
<code>getNPCPositions</code>	Devuelve la lista de posiciones de los NPCs
<code>getMovablePositions</code>	Devuelve la lista de posiciones de objetos móviles
<code>getImmovablePositions</code>	Devuelve la lista de posiciones de objetos inmóviles
<code>getResourcesPositions</code>	Devuelve la lista de posiciones de recursos
<code>getPortalsPositions</code>	Devuelve la lista de posiciones de los portales

Estas llamadas suelen ser costosas. Intentad minimizarlas.

Nota: Revisad el tutorial de GVG-AI que adjuntamos con la práctica. Contiene esta información y mucha más.

```
@Override
public ACTIONS act(StateObservation stateObs, ElapsedCpuTimer elapsedTimer) {
    //Posicion del avatar
    Vector2d avatar = new Vector2d(stateObs.getAvatarPosition().x / fescala.x,
        stateObs.getAvatarPosition().y / fescala.y);

    //Probamos las cuatro acciones y calculamos la distancia del nuevo estado al portal.
    Vector2d newPos_up = avatar, newPos_down = avatar, newPos_left = avatar, newPos_right = avatar;
    if (avatar.y - 1 >= 0) {
        newPos_up = new Vector2d(avatar.x, avatar.y-1);
    }
    if (avatar.y + 1 <= stateObs.getObservationGrid()[0].length-1) {
        newPos_down = new Vector2d(avatar.x, avatar.y+1);
    }
    if (avatar.x - 1 >= 0) {
        newPos_left = new Vector2d(avatar.x - 1, avatar.y);
    }
    if (avatar.x + 1 <= stateObs.getObservationGrid().length - 1) {
        newPos_right = new Vector2d(avatar.x + 1, avatar.y);
    }

    //Manhattan distance
    ArrayList<Integer> distances = new ArrayList<Integer>();
    distances.add((int) (Math.abs(newPos_up.x - portal.x) + Math.abs(newPos_up.y-portal.y)));
    distances.add((int) (Math.abs(newPos_down.x - portal.x) + Math.abs(newPos_down.y-portal.y)));
    distances.add((int) (Math.abs(newPos_left.x - portal.x) + Math.abs(newPos_left.y-portal.y)));
    distances.add((int) (Math.abs(newPos_right.x - portal.x) + Math.abs(newPos_right.y-portal.y)));

    // Nos quedamos con el menor y tomamos esa accion.
    int minIndex = distances.indexOf(Collections.min(distances));
    switch (minIndex) {
        case 0:
            return Types.ACTIONS.ACTION_UP;
        case 1:
            return Types.ACTIONS.ACTION_DOWN;
        case 2:
            return Types.ACTIONS.ACTION_LEFT;
        case 3:
            return Types.ACTIONS.ACTION_RIGHT;
        default:
            return Types.ACTIONS.ACTION_NIL;
    }
}
```

En esencia:

- 1) Calculamos la posición del avatar en coordenadas grid.
- 2) Vemos a dónde se movería con todas y cada una de las 4 acciones posibles.
Nota: este código no tiene en cuenta las casillas prohibidas (muros), y siempre calcula y añade la distancia (aunque se corresponda con una casilla intransitable).
- 3) Calculamos la distancia de las nuevas posiciones al portal.
- 4) Escogemos la acción asociada con la distancia más corta. Es decir, escogemos la acción que más nos acerca de modo inmediato al objetivo.

La Práctica 1 consiste en **desarrollar cuatro controladores basados en técnicas deliberativas de búsqueda dentro del entorno GVGAI que guíen a un avatar a resolver** un juego en distintos niveles. El juego escogido es el **Catapults (modificado)** disponible con el **índice 122** en los tipos de juego "singleplayer" que se pueden encontrar en el **fichero "all_games_sp.csv"**
(tras incorporar el material adicional disponible en PRADO)

4 acciones:

- Izquierda
- Derecha
- Arriba
- Abajo

4 tipos de casilla:

- **Avatar**
- **Catapultas**
- **Monedas**
- **Obstáculos / Plantas**
- **Llave**
- **Objetivo / Meta**
- **Agua, que se puede traspasar en modo murciélago con el uso de una catapulta**



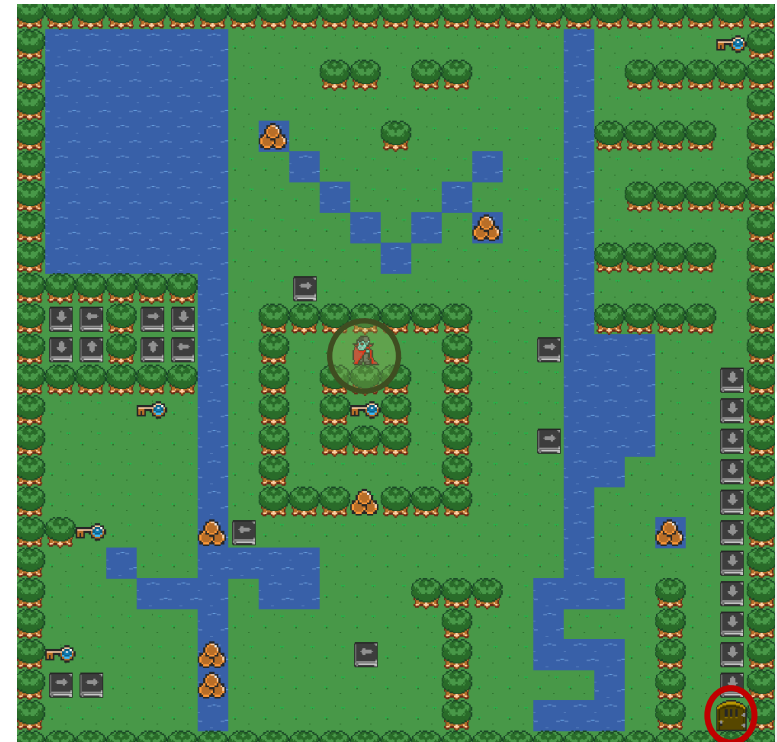
El objetivo de la práctica es que los estudiantes refuercen conocimientos y se familiaricen con **técnicas deliberativas de búsqueda**.

Para ello, el avatar deberá resolver varios mapas de progresiva dificultad con los siguientes algoritmos:

1. **Búsqueda no informada (offline) con Búsqueda en Profundidad**
2. **Búsqueda heurística (offline) con A***
3. **Búsqueda heurística (online) en tiempo real con RTA***
4. **Búsqueda heurística (online) en tiempo real con LRTA* (k)**

Todos estos algoritmos se repasarán en las próximas sesiones de prácticas

Los mapas son proporcionados por los profesores de prácticas, aunque **es necesario** que los estudiantes creen sus propios mapas y verifiquen el comportamiento de sus algoritmos en ellos, ya que la evaluación se realizará en mapas distintos (aunque de dificultad similar).



1) Test.java

```
String AStarController =  
"tracks.singlePlayer.evaluacion.src_APELLIDO1_APELLIDO2_NOMBRE.Ag  
enteAStar";
```

...

```
ArcadeMachine.runOneGame(game, level1, visuals, AStarController,  
recordActionsFile, seed, 0);
```

2) En el primer tick, GVGAI automáticamente llama al constructor de AgenteAStar (en donde se inicializa la posición de los recursos y casillas prohibidas, etc.) y luego llama al método act().

3) En esta primera llamada al act(), como la ruta todavía no existe, esta se calcula a partir del nodo inicial, el nodo final y las casillas prohibidas (utilizando el algoritmo correspondiente; en este caso, A*), y se ejecuta la primera acción:

```
return myaction;
```

4) Todas las llamadas restantes a act() se limitan a ejecutar/devolver las acciones restantes en la ruta calculada.

Añadir juegos, mapas y plantilla

Puesto que el equipo docente ha creado versiones modificadas de los juegos es necesario añadirlos a la instalación base. Debe:

1. Descargar las definiciones y mapas de entrenamiento disponibles en PRADO.
2. Todos los archivos .txt, que contienen los mapas y las definiciones deben colocarse en

<directorio>/examples/gridphysics

3. El archivo all_games_sp.csv debe sustituir al archivo que se encuentra en

<directorio>/examples/all_games_sp.csv

Una vez realizado esto, los juegos de este curso estarán disponibles con los códigos 122 (Práctica 1) y 123 (Competición).

Añadir juegos, mapas y plantilla

El último paso para la preparación del entorno es crear el paquete donde trabajará el alumnado y añadir la clase MetricsProvider.java para entregar las métricas adicionales de ejecución.

1. Extraer la clase **MetricsProvider**, contenida en el mismo zip disponible en PRADO (y que también contiene las plantillas y mapas de entrenamiento).

2. Colocar la clase MetricsProvider.java en

<directorio>/src/tracks/singlePlayer/MetricsProvider.java

3. Colocar el paquete de evaluación de alumno en

**<directorio>/src/tracks/singlePlayer/evaluacion/
src_APELLIDO1_APELLIDO2_NOMBRE**

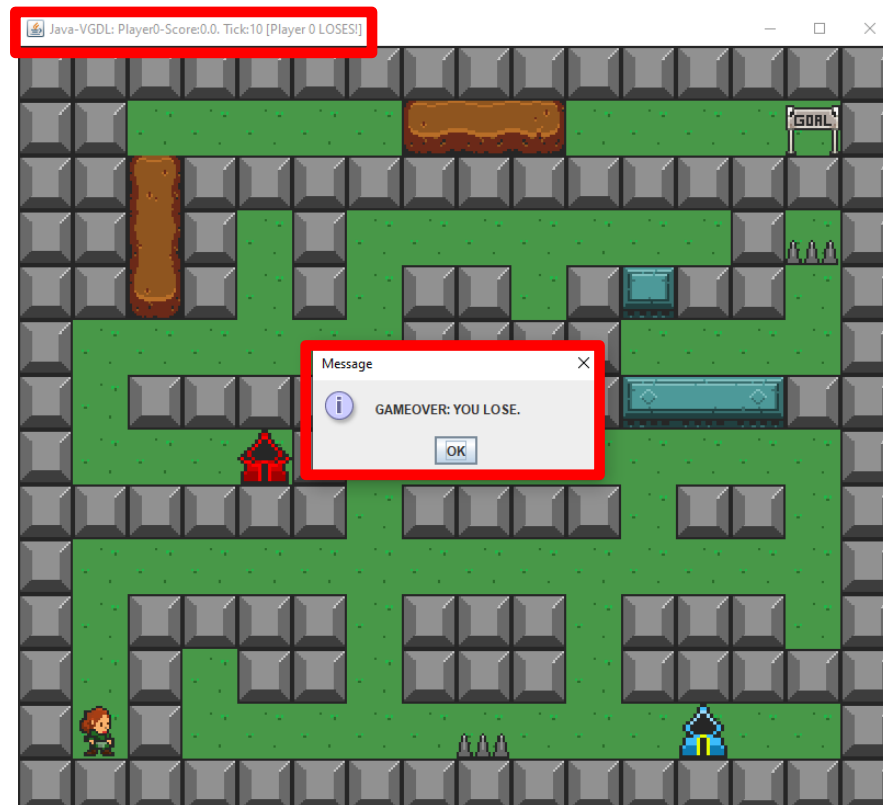
y actualizar el nombre del paquete con su correspondiente nombre y apellidos.

- La clase **MetricsProvider.java** es una clase de tipo *singleton* que se debe utilizar para fijar todas las métricas complementarias (tamaño de la ruta, nodos explorados, nodos en abiertos y cerrados, etc.) que deben entregarse para el **20% de la calificación de cada algoritmo**. Un ejemplo de uso de esta clase para fijar el número de nodos expandidos a 1000 sería:

```
MetricsProvider.getInstance().setNodosExpandidos(1000);
```

- Las métricas serán recogidas tras finalizar el método que ejecuta el juego (*ArcadeMachine.runOneGame*) con el agente en un archivo de **Test.java** creado por el profesorado. Es decir, **no se tendrán en cuenta los elementos mostrados por salida estándar**.

- Se debe **modificar el número máximo de timesteps**, tanto en la definición del juego como en el entorno de GVGA.
 - Los *timesteps* (o *ticks*) representan el tiempo máximo permitido para jugar. Si se alcanza ese valor, y no se ha conseguido el objetivo del juego, se pierde la partida.
 - Ejemplo con 10 timesteps:



- El número de *timesteps* se debe incrementar a **5.000**.
- Para modificarlo en vuestro ordenador:
 - En `src/core/competition/CompetitionParameters.java`:

```
public static final int MAX_TIMESTEPS = 5000;
```



- Todos los algoritmos propuestos tienen un **comportamiento determinista**.
 - Todas las **implementaciones correctas** proporcionarán los **mismos resultados** (mismas acciones, mismo número de nodos expandidos), a excepción del tiempo de ejecución, que variará de una máquina (e implementación) a otra.
- Para ello **SIEMPRE** se **expandirán** los vecinos de un nodo en el siguiente **orden**:
 DERECHA, ARRIBA, IZQUIERDA, ABAJO
- Algunos algoritmos visitan los nodos usando información heurística.
 - Por ejemplo, en A* los nodos se visitan según el valor de $f(n)$, desempataando por $h(n)$. ¿Y si también hay un empate en $h(n)$?
 - Se dará prioridad a los nodos más antiguos de la lista de abiertos.
 - Para los vecinos de un nodo, **se usará el orden de expansión arriba indicado para cualquier desempate final**.

- GVGAI **descalifica** al agente si:
 - El tiempo empleado en el **constructor** es mayor que 1s.
 - El tiempo empleado en el **método "act"** es mayor que 50ms.
 - Es decir, cada tick, el motor del juego llama a act, y si el return de act tarda en llegar más de 50ms → pierdes!
 - Si el tiempo de act está en [40,50]ms, se ignora la acción calculada y realiza un ACTIONS.NIL.
- En la práctica **NO se permite** el uso del **constructor** para **ninguna operación** relacionada directamente con los **algoritmos de búsqueda**. Todo el algoritmo de búsqueda se realizará en el método "act".
 - **En el constructor sí se podría inicializar alguna variable** (factor de escala, posición de la casilla objetivo, casillas prohibidas, etc.) **o tarea menor** que no tenga relación con el algoritmo de búsqueda propiamente dicho.
- Por tanto, **una estructura de datos eficiente** es crucial para el correcto funcionamiento de los agentes.

- Pasos a seguir:
 1. Descargar e instalar el entorno GVGAi.
 2. Seguir las indicaciones anteriores para probar varios juegos y niveles.
 3. Consultar y revisar los materiales proporcionados con la práctica:
 - Esta presentación
 - Enunciado de la práctica (y la competición)
 - Tutorial sobre GVGAi
 - Consultar la documentación sobre el código en caso de que sea necesario.
 - *La estructura del código y documentación básica está en <https://github.com/EssexUniversityMCTS/gvgai/wiki/Code-Structure>*
 4. Explorar y ejecutar el juego de la carrera de camellos (Camel Race), del que se proporciona un script sencillo (y manifiestamente mejorable).
 5. Comenzar la práctica: se recomienda implementar los algoritmos en el orden propuesto y hacer uso de la plantilla disponible en PRADO.

- La **temporización sugerida** para esta práctica:
 - *Semana 9 de marzo*: Presentación Práctica 1.
 - Instalación de GVGAI, revisar materiales, ejecutar códigos de prueba (Camel Race).
 - *Semana 16 de marzo*: Presentación búsqueda offline.
 - Implementación de Búsqueda en Profundidad y A*.
 - *Semana 23 de marzo*: Presentación búsqueda online.
 - Implementación de RTA* y LRTA*(k).
 - *Semana 30 de marzo*: Semana santa (Vacaciones).
 - *Semana 6 de abril*: Seguimiento y resolución de dudas.
 - Preparación de la memoria y Competición (para quien quiera participar en ella).

Entrega P1: 12 de abril de 2026 hasta las 23:59

- Para el desarrollo de la práctica, cada estudiante debe crear el siguiente **Java package**:

`src/tracks/singlePlayer/evaluacion/src_APELLIDO1_APELLIDO2_NOMBRE`

- Dentro de este paquete se crearán, al menos, las siguientes **clases Java**:
 - `AgenteProfundidad`
 - `AgenteAStar`
 - `AgenteRTAStar`
 - `AgenteLRTAStarK`
 - `AgenteCompeticion` (solo si se participa en la competición)
- Se pueden incluir otras clases auxiliares dentro del paquete que sean utilizadas por las anteriores.
- Los nombres de clase también son case-sensitive, por lo que se deben respetar las mayúsculas/minúsculas.

!!!Es **REQUISITO IMPRESCINDIBLE** respetar esta nomenclatura!!!

- El material a **entregar** a través de PRADO será un **fichero ZIP** con el siguiente contenido:
 - Un fichero **PDF** con la **memoria (máx 5 páginas)**.
 - Otro fichero **PDF** si se ha usado IA generativa con los *prompts* y respuestas, explicando el uso dado a las herramientas de IA empleadas.
 - La **carpeta** del **paquete Java** anteriormente descrito, teniendo en cuenta que **el código debe estar bien comentado**.
- **Fecha límite de entrega:**
 - **12 de abril a las 23:59 horas**

NOTA: se recomienda encarecidamente **revisar las restricciones de la entrega en el enunciado de la práctica**, y seguir escrupulosamente todas las indicaciones allí incluidas.